

Day 5: Real Hardware: Transpilation, Noise, and Error Mitigation

SC Quantathon v3 Bootcamp
Clemson Quantum Club

Every earlier day ended with an optional cell that sent a circuit to a real processor and came back with a histogram slightly different from the simulator's. Today that difference is the subject. These notes follow the Day 5 notebook section by section: what a real machine is, how a circuit is rewritten to run on it, where each kind of error comes from, and two ways to get some of the lost accuracy back. Everything runs against **FakeSherbrooke**, a snapshot of `ibm_sherbrooke`, a 127-qubit Eagle processor retired in July 2025, that carries its real calibration data and runs offline; the live machines today are Heron and Nighthawk processors, and everything the snapshot teaches carries over.

1 Meet the machine

A superconducting processor is a chip of transmon qubits held near 15 mK and driven by microwave pulses. Three facts about it decide what a circuit can do, and all three are attached to the backend object.

- The *coupling map*: which pairs of qubits are physically connected. Sherbrooke has 127 qubits joined by 144 couplers in a heavy-hexagon lattice; no qubit touches more than three neighbours, and most touch two. The sparse lattice is a deliberate trade: fewer neighbours means fewer frequency collisions and less crosstalk between qubits that are driven at the same time, at the price of longer routes for circuits that want to connect distant qubits.
- The *native gates*: the operations the electronics can play. On Sherbrooke they are R_z , $\sqrt{X}$ (`sx`), X , and one two-qubit gate, the echoed cross-resonance gate `ecr`, which is a CNOT up to single-qubit corrections.
- The *calibration data*: how good each qubit and each gate currently is, remeasured every few hours.

Each qubit has a T_1 , the time for an excited $|1\rangle$ to decay to $|0\rangle$; a T_2 , the time for a superposition to lose its phase; and a *readout error*, the chance that a measurement reports the wrong bit. Each coupler has a gate error and a duration. Table 1 gives the snapshot's medians, and Figure 1 the spread across the chip.

Two lessons hide in the table. Calibration data are measurements, not specifications: a few qubits report $T_2 > 2T_1$, which physics forbids, and a few qubits and couplers report errors of 50% or 100%, which means “currently unusable” rather than a number to plug in. And the time scales are separated by two orders of magnitude: a two-qubit gate takes half a microsecond against coherence times of hundreds of microseconds, so a circuit of depth twenty runs in about ten microseconds and loses only a few percent to decoherence. For short circuits the two-qubit gate error, at roughly a percent per gate, matters first.

quantity	median	notes
T_1	278 μs	qubit 0: 382 μs
T_2	170 μs	qubit 0: 132 μs
readout error	1.98%	one qubit at 50%, effectively dead
ecr gate error	0.78%	best pair (61, 60) at 0.35%; nine pairs at 100%
ecr duration	533 ns	single-qubit gates are tens of ns

Table 1: Calibration data of the `ibm_sherbrooke` snapshot.

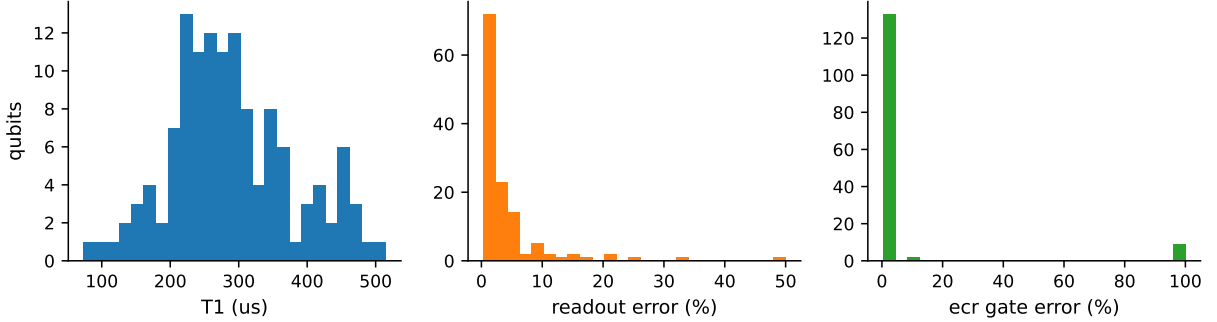


Figure 1: Calibration data across the 127 qubits and 144 couplers of the snapshot: T_1 , readout error, and ecr gate error.

2 Transpilation

The circuits you write use H , CNOT, and whatever else is convenient, on qubits numbered from zero. The chip runs only its native gates, only between coupled pairs, on physical qubits of its choosing. The *transpiler* bridges the gap with three jobs, which Qiskit runs as layout, routing, translation, and then an optimization pass:

1. choose a *layout* of the circuit’s qubits onto physical qubits, preferring good ones;
2. *route*, inserting SWAP gates wherever a two-qubit gate needs a pair that is not coupled, each SWAP costing three two-qubit gates;
3. *translate* every gate into native ones, using identities such as $H = R_z(\frac{\pi}{2})\sqrt{X}R_z(\frac{\pi}{2})$ up to phase.

The result is an *ISA circuit* (instruction set architecture), the only kind a real backend accepts. The Bell circuit’s one H and one CNOT become seven R_z , four $\sqrt{X}$, and one ecr, and the depth grows from 3 to 8; the two logical qubits land on physical qubits 0 and 1.

A few circuit tools help in reading the result. A *barrier* is a visual divider that also stops the optimizer from merging gates across it. `measure(q, c)` stores one qubit into one bit of a classical register you declared; `measure_all()` adds a register named `meas` and measures every qubit. `depth()` counts the layers of gates that cannot run at the same time, and `count_ops()` counts gates by name. Figure 2 shows the translation step alone on the one-qubit circuit H, T, H : three gates become five, of the kinds R_z and $\sqrt{X}$ only, and `Operator.equiv` confirms the two circuits are the same unitary up to a global phase. The notebook’s version of the same cell keeps a barrier between T and the second H , which stops the translator from merging two adjacent R_z gates, so it reports four R_z instead of three: a barrier costs nothing on the chip but can cost an optimization.

Global Phase: $13\pi/8$

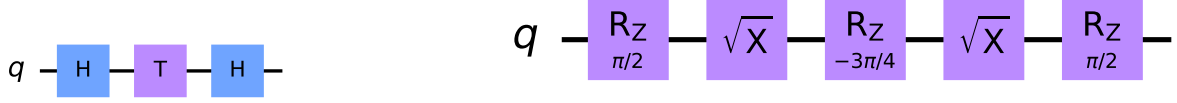


Figure 2: The circuit H, T, H as written (left, depth 3) and translated to IBM’s native gate set (right, depth 5: three R_z and two $\sqrt{X}$).

The *optimization level*, 0 to 3, controls how hard the transpiler works to shorten the result by cancelling adjacent gates, merging rotations, and searching for a better layout. Table 2 shows a five-qubit GHZ circuit at each level. Level 0 does the minimum; level 1 is the fastest and, for this circuit, also the shortest, because the linear chain of CNOTs already fits the lattice; levels 2 and 3 choose a different layout and land on a slightly longer circuit, so a higher level is not guaranteed to be shorter. The number of two-qubit gates is the figure to watch, since those carry most of the error.

optimization level	depth	ecr gates	all gates
0	32	4	71
1	17	4	37
2	25	4	48
3	25	4	48

Table 2: A five-qubit GHZ circuit (1 H , 4 CNOTs) transpiled for the snapshot at each optimization level.

Routing shows its cost on a machine whose lattice does not match the circuit. **FakeManilaV2** is five qubits in a line, 0–1–2–3–4. A star circuit with CNOTs from qubit 0 to each of the others cannot run directly, because qubit 0 touches only qubit 1. At level 3 the transpiler produces 10 native CNOTs for the 4 logical ones and a depth of 14: the extra six are two SWAPs. On a real job, routing overhead of this kind is often the single largest source of error, which is why the layout step tries so hard to avoid it. The arithmetic is unforgiving: a SWAP is three CNOTs (Day 4), so at a percent of error per two-qubit gate each routed SWAP costs about three percent of fidelity, and a circuit that needs ten of them has lost a quarter of its signal before any of its own gates have run. Layout is therefore a search: the transpiler tries to find a placement of the logical qubits on the coupling map under which every two-qubit gate already sits on a coupler, and only when no such placement exists does it route.

3 Where errors come from

Three mechanisms account for nearly everything that separates a real run from the simulator. Each has a simple model, and `qiskit_aer.noise` can attach any of them to a circuit, so each can be studied alone before they are mixed.

3.1 Decoherence

An excited qubit relaxes to its ground state at a constant rate, so after an idle time t the state $|1\rangle$ has survived with probability

$$P(1) = e^{-t/T_1}. \quad (1)$$

A superposition also loses its phase. The relative phase of $|+\rangle$ drifts randomly, and averaging over the drift shrinks the x component of the Bloch vector by e^{-t/T_2} , so a measurement in the X basis returns $+$ with probability

$$P(+) = \frac{1}{2}(1 + e^{-t/T_2}), \quad (2)$$

which decays from 1 toward $\frac{1}{2}$: a fully dephased qubit is a fair coin in every basis. Both curves come from the same assumption, a constant rate: if an excited qubit decays with probability dt/T_1 in every short interval dt , then $dP/dt = -P/T_1$ and $P(t) = e^{-t/T_1}$, the same law as radioactive decay. Relaxation also destroys phase, and it does so at half the rate: the off-diagonal element of the density matrix that carries the coherence decays as $e^{-t/2T_1}$ from relaxation alone, so $1/T_2 = 1/(2T_1) + 1/T_\phi$ with T_ϕ the pure dephasing time, and $T_2 \leq 2T_1$ with equality only when nothing but relaxation is happening. Figure 3 is both curves for qubit 0 of the snapshot, produced by the `thermal_relaxation_error` channel with that qubit's T_1 and T_2 .

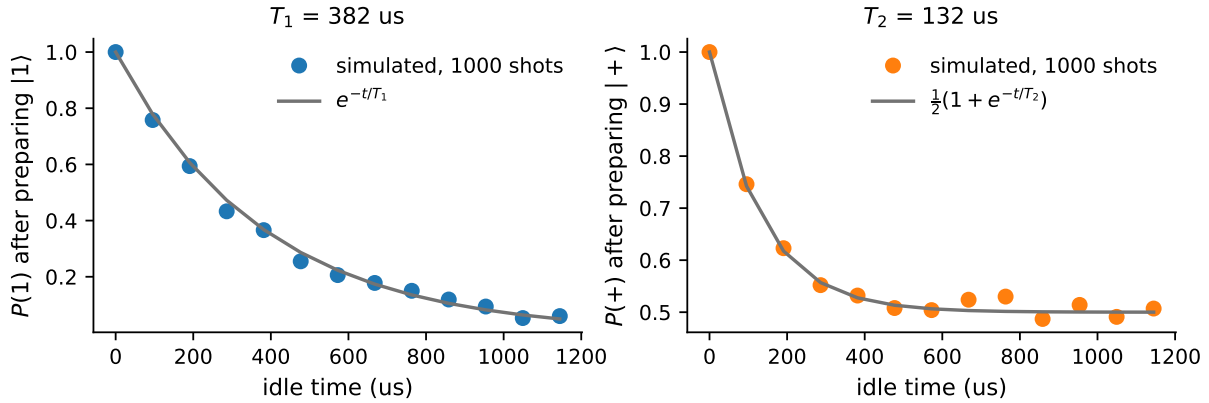


Figure 3: Decoherence of qubit 0 of the snapshot, simulated with its calibrated $T_1 = 382 \mu\text{s}$ and $T_2 = 132 \mu\text{s}$. Left: survival of $|1\rangle$, Eq. (1). Right: survival of the phase of $|+\rangle$, Eq. (2).

3.2 Gate error

Each gate is a pulse that is slightly wrong. The simplest model is *depolarizing* noise: after each gate, with probability p the state is replaced by the maximally mixed state, which is equally likely to be anything. After k gates the chance that no scrambling has occurred is $(1-p)^k$, and a scrambled two-qubit state still reads 00 a quarter of the time, so a chain of k CNOTs, which is the identity on paper when k is even, returns 00 with probability

$$P(00) = \frac{1}{4} + \frac{3}{4}(1-p)^k. \quad (3)$$

Where the quarter comes from: the depolarizing channel maps the two-qubit density matrix to $\rho \mapsto (1-p)\rho + pI/4$, and applying it k times gives $\rho_k = (1-p)^k\rho + (1-(1-p)^k)I/4$, since the maximally mixed state is a fixed point. Reading off $\langle 00|\rho_k|00\rangle$ with $\rho = |00\rangle\langle 00|$ gives $(1-p)^k + \frac{1}{4}(1-(1-p)^k)$, which is Eq. (3). Figure 4 tests it at $p = 1\%$: the formula predicts 0.863 and 0.752 at $k = 20$ and 40, and the simulation measures 0.866 and 0.754. This is a gate-repetition experiment. The standard hardware method, *randomized benchmarking*, applies sequences of random Clifford gates followed by an inverting gate, which averages coherent errors into exactly this depolarizing decay.

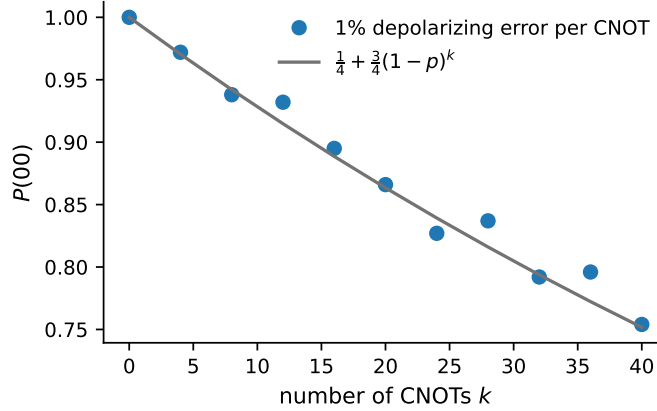


Figure 4: $P(00)$ after k CNOTs with 1% depolarizing error per gate, and Eq. (3).

3.3 Readout error

The last step has its own error: a $|0\rangle$ is sometimes reported as 1 and a $|1\rangle$ as 0, at rates that differ. The two rates make a 2×2 *confusion matrix* whose columns are the prepared state and whose rows are the reported one. For one qubit the matrix is

$$M = \begin{pmatrix} 1 - \epsilon_0 & \epsilon_1 \\ \epsilon_0 & 1 - \epsilon_1 \end{pmatrix}, \quad (4)$$

with ϵ_0 the chance of reading 1 when $|0\rangle$ was prepared and ϵ_1 the reverse; on the snapshot the two are typically a percent or two each and unequal, since a $|1\rangle$ can also decay during the readout pulse. When the qubits read out independently, the two-qubit matrix is the tensor product $M_1 \otimes M_0$ of the single-qubit ones. For a 50/50 state a symmetric error ($\epsilon_0 = \epsilon_1$) moves as many counts one way as the other and changes nothing, which is why Day 1’s random-bit histogram showed no sign of it; only the asymmetry shifts the counts. Readout error is the easiest to correct, because it acts after the circuit and its matrix can be measured directly.

4 Simulating the machine

`AerSimulator.from_backend` assembles all three mechanisms from the calibration data, thermal relaxation on every gate, depolarizing error at the measured rates, and each qubit’s readout error, and runs an ISA circuit through them. It is the best offline prediction of what the machine will do. The prediction has limits worth knowing: the model treats every error as independent and memoryless, so it misses crosstalk between simultaneous gates, coherent errors that add up in phase rather than at random, and drift between the calibration and the run, which together explain why a real device is usually somewhat worse than its snapshot. Table 3 and Figure 5 compare the ideal simulator, the noisy simulator, and Day 4’s recorded run of the same Bell circuit on `ibm_kingston`, a Heron r2 processor whose two-qubit gate is `cz`, on physical qubits 55 and 59. The noisy simulator predicts 3.0% impossible outcomes and the device showed 1.0%; the Hellinger fidelities to the ideal histogram are 0.970 and 0.990. The two agree on the scale of the error, a percent or so, and not on the number, and the direction is the informative part: the snapshot is of an Eagle machine retired in 2025, while the live run used a current processor on a pair chosen from that day’s calibration table, so the device did better than the old snapshot predicts. A snapshot of the same machine on the same day would be expected to land closer.

	counts (00, 01, 10, 11)	impossible outcomes	fidelity to ideal
ideal simulator	506, 0, 0, 494	0%	1.000
noisy simulator	504, 13, 17, 466	3.0%	0.970
ibm_kingston, Day 4 run	512, 6, 4, 478	1.0%	0.990

Table 3: The Bell circuit three ways.

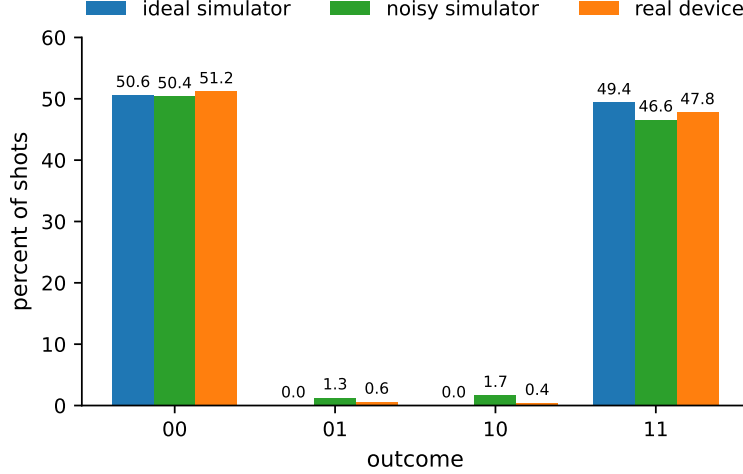


Figure 5: The Bell circuit on the ideal simulator, on the noisy simulation of the snapshot, and on ibm_kingston (Day 4’s run).

5 Readout error mitigation

Readout error acts after the circuit, so linear algebra can undo it. Prepare each basis state in turn on the same physical qubits the circuit uses, measure it, and record the distribution of reported strings. That fills a matrix M whose entry M_{ij} is the probability of reading i when j was prepared. A measured distribution is then $\vec{p}_{\text{meas}} = M\vec{p}_{\text{true}}$, and

$$\vec{p}_{\text{true}} = M^{-1}\vec{p}_{\text{meas}} \quad (5)$$

recovers the distribution the circuit would have produced with perfect readout. For two qubits M is 4×4 and needs four calibration circuits. Statistical noise can push entries of the solution slightly negative; clipping and renormalizing is the standard fix.

For the Bell circuit on physical qubits 0 and 1 of the snapshot, the measured M has diagonal entries 0.964, 0.965, 0.966, 0.965, and applying Eq. (5) to the noisy counts drops the impossible outcomes from 3.0% to 0.0% (Figure 6). Inversion has a price. M^{-1} has entries slightly larger than one on the diagonal and small negative entries off it, so it amplifies the statistical noise of the counts by a few percent for a matrix like this one, and by much more when readout is poor and M is far from the identity. It also scales badly: a full M for n qubits is $2^n \times 2^n$ and needs 2^n calibration circuits, so beyond a handful of qubits the per-qubit form of Eq. (4) is measured instead and the tensor product is assumed. What survives mitigation is the error that happened *inside* the circuit, mostly the two-qubit gate, which this method cannot touch. On the runtime, `EstimatorV2` with `resilience_level = 1` performs this calibration automatically, using a technique called TREX.

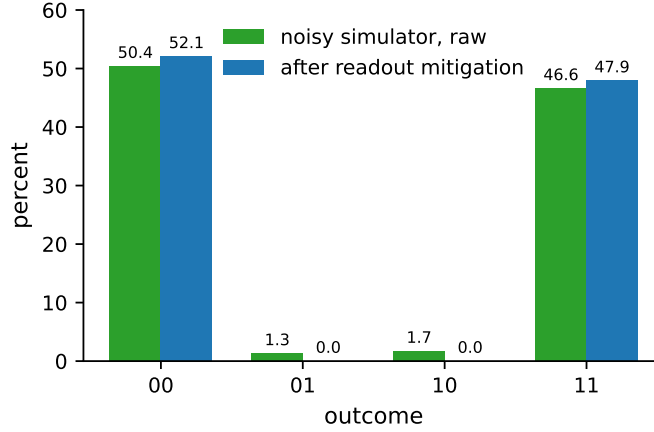


Figure 6: The noisy simulator’s Bell counts before and after readout error mitigation with a measured confusion matrix.

6 Zero-noise extrapolation

Gate error cannot be calibrated away, but it can be amplified on purpose and extrapolated out. Replace every CNOT by three CNOTs, the same gate on paper and three times the gate error in practice, then by five. An expectation value measured at noise scales $\lambda = 1, 3, 5$ traces a curve, and the value of that curve at $\lambda = 0$ estimates the noiseless result. This is *zero-noise extrapolation* (ZNE). For $\langle ZZ \rangle$ of the Bell state, exactly 1 without noise, a CNOT with 3% depolarizing error gives 0.9677, 0.9100, and 0.8603 at the three scales (Figure 7). The simplest version uses two points: if the error is linear in λ , then $E(\lambda) = E_0 + a\lambda$, and the measurements at $\lambda = 1$ and 3 give $E_0 = (3E_1 - E_3)/2$, which here is $(3 \times 0.9677 - 0.9100)/2 = 0.997$. A straight line fitted through all three points reaches 0.9932 at $\lambda = 0$ and a quadratic reaches 0.9995, against the unmitigated 0.9677. Folding works because a CNOT is its own inverse, so $\text{CNOT}^3 = \text{CNOT}$ exactly on paper; for a general gate G the fold is $G G^\dagger G$, the same identity. Both recover most of the lost 0.03; the quadratic lands within a thousandth of the true value because the exact decay, $(1 - p)^\lambda$ up to the $\frac{1}{4}$ offset, is curved, and a fit that can bend follows it better. An extrapolation inherits the scatter of its inputs, so with fewer shots either fit can also overshoot 1.

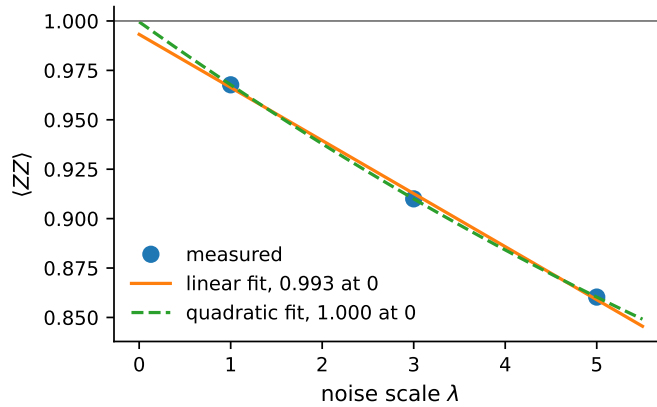


Figure 7: Zero-noise extrapolation of $\langle ZZ \rangle$ for the Bell state under 3% depolarizing CNOT error. The three points are measured at noise scales 1, 3, and 5; the fits are read off at scale 0.

ZNE costs three runs instead of one and returns an extrapolation, not a measurement, so its error bars are larger and it can overshoot. Its strength is that it needs no model of the noise. On the runtime, `EstimatorV2` with `options.resilience.zne_mitigation = True` does the folding and fitting itself.

7 The live machine

Everything above used the snapshot. The notebook’s last section sends the same Bell circuit to the least busy live processor, which was again `ibm_kingston` on physical qubits 55 and 59, and this time it sends two jobs. The first is a Sampler job of five circuits: the Bell circuit and the four calibration circuits of Section 5, transpiled onto the same pair, so that the confusion matrix is measured on the qubits that actually ran rather than borrowed from the snapshot. The second is an Estimator job for $\langle ZZ \rangle$ with the runtime’s own readout mitigation (TREX) and zero-noise extrapolation switched on, Sections 5 and 6 done by the service. Table 4 and Figure 8 hold the results.

	value	note
raw counts (00, 01, 10, 11)	527, 6, 4, 463	1.0% impossible, fidelity 0.989
confusion matrix diagonal on the pair	0.999, 0.985, 0.995, 0.992	readout error 0.1% to 1.5%
readout-mitigated probabilities	0.527, 0.005, 0.001, 0.467	
$\langle ZZ \rangle$, exact	1.0000	
$\langle ZZ \rangle$, snapshot ZNE (Section 6)	0.9995	quadratic fit
$\langle ZZ \rangle$, device raw	0.9800	from the raw counts
$\langle ZZ \rangle$, device readout mitigated	0.9884	with the on-device matrix
$\langle ZZ \rangle$, device with TREX and ZNE	0.9952	the runtime Estimator

Table 4: The Bell circuit on `ibm_kingston`, physical qubits 55 and 59, 1000 shots per circuit.

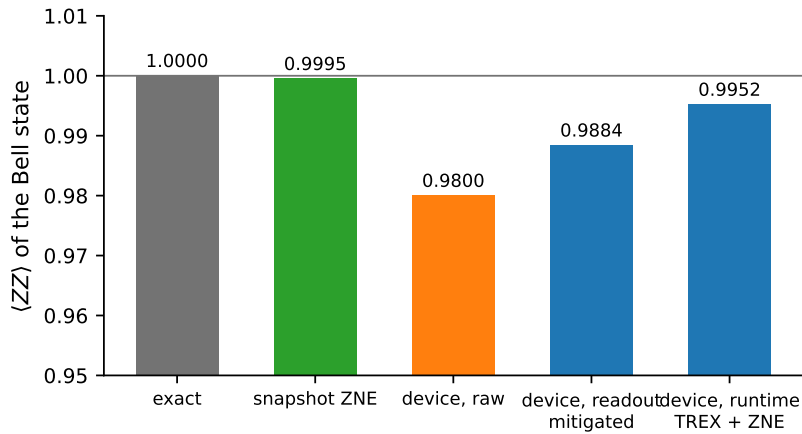


Figure 8: $\langle ZZ \rangle$ of the Bell state, exact and as estimated five ways; the axis starts at 0.95.

Two things are worth reading off. Readout mitigation barely moves this run, because the pair’s readout is already at the percent level, so the ten impossible shots and the tilt between 00 and 11 (527 against 463, about two σ) came from inside the circuit: the `cz` gate and relaxation during it. That is exactly what the readout matrix cannot touch, and exactly what zero-noise extrapolation is for: the runtime’s TREX plus ZNE takes $\langle ZZ \rangle$ from 0.980 to 0.995, three quarters of the way to

the exact value, and it did so by running folded copies of the circuit and fitting, as Section 6 did on the snapshot. The remaining half percent is the price of extrapolating from noisy points. A second lesson is in the pair itself: the same two qubits gave 1.0% impossible outcomes on Day 4 and 1.0% here, an hour apart, while qubit 0 of the same chip was misreading half its zeros on Days 1 and 2. Which qubits a job lands on matters more than which machine.

8 Summary

- A backend carries a coupling map, a native gate set, and calibration data: T_1 , T_2 , and readout error per qubit; error and duration per coupler. Fake backends are offline snapshots with the same data, and a few qubits in any snapshot are effectively dead.
- The transpiler lays out logical qubits onto physical ones, routes with SWAPs where the lattice requires, and translates gates to native ones. Two-qubit gate count is the cost that matters. Only ISA circuits run on hardware.
- $|1\rangle$ survives as e^{-t/T_1} and a superposition keeps its phase as e^{-t/T_2} , Eqs. (1) and (2). Depolarizing gate error compounds as $(1 - p)^k$, Eq. (3). Readout error is a confusion matrix applied after the circuit.
- `AerSimulator.from_backend` combines the three from calibration data; for the Bell state its impossible-outcome fraction lands within two percent of the device’s, with the current processor beating the retired snapshot (Table 3).
- Readout mitigation inverts the measured confusion matrix, Eq. (5). Zero-noise extrapolation folds gates to amplify noise and extrapolates to zero. The runtime Estimator offers both as options.
- On the live machine the readout matrix measured on the pair removed little, because readout was already at the percent level; the runtime’s TREX plus ZNE moved $\langle ZZ \rangle$ from 0.980 to 0.995 (Table 4).

References

- [1] IBM Quantum documentation, *Introduction to transpilation*. <https://quantum.cloud.ibm.com/docs/en/guides/transpile>
- [2] IBM Quantum documentation, *Error mitigation and suppression techniques*. <https://quantum.cloud.ibm.com/docs/en/guides/error-mitigation-and-suppression-techniques>
- [3] IBM Quantum documentation, *Building noise models with Qiskit Aer*. <https://quantum.cloud.ibm.com/docs/en/guides/build-noise-models>
- [4] P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver, “A quantum engineer’s guide to superconducting qubits,” *Applied Physics Reviews* **6**, 021318 (2019).
- [5] K. Temme, S. Bravyi, and J. M. Gambetta, “Error mitigation for short-depth quantum circuits,” *Phys. Rev. Lett.* **119**, 180509 (2017).

- [6] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary ed. (Cambridge University Press, 2010), chapter 8, for the channels behind Eqs. (1) to (3).